

RL in Quantum Applications

From control pulses to feedback, error correction, and quantum agents

FYS4411 – Reinforcement Learning and Agentic AI

University of Oslo, Department of Physics

Based on Bukov and Marquardt, *Reinforcement Learning for Quantum Technology*,
arXiv:2601.18953

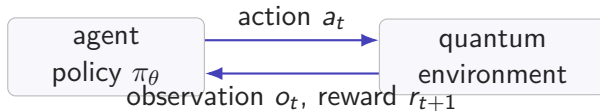
What this lecture is about

A guided map of where reinforcement learning is useful in quantum technology.

- ▶ Translate quantum protocols into states, observations, actions, rewards, and episodes.
- ▶ Separate classical RL for quantum systems from quantum-enhanced and fully quantum RL.
- ▶ Study state preparation, gate design, circuit design, feedback, error correction, and metrology.
- ▶ Tie algorithms to the physics: observation model, time scale, reward access, hardware noise, and sample budget.

Why RL enters quantum technology

The common structure is sequential interaction with a device or simulator.



Quantum task	Sequential decision
Control	choose amplitudes $u_k(t)$ over a finite horizon
Gates	choose pulse segments or calibration updates
Circuits	append, remove, route, or rewrite gates
Feedback	adapt after measurement outcomes
QEC	infer and correct errors from syndrome histories

The quantum device is the environment

In most applications the agent is classical; the system it controls is quantum.

- ▶ The agent does not see the wavefunction in an experiment.
- ▶ It sees classical data: bit strings, counts, homodyne records, or syndromes.
- ▶ The policy maps available records to the next experimental choice.
- ▶ Training can use a simulator, real hardware, or both.

Policy interface

$$h_t = (o_1, a_1, \dots, o_t), \quad a_t \sim \pi_\theta(\cdot \mid h_t).$$

The application families in the review

They differ mainly in what the action does to the quantum system.

Application	Action	Reward signal	Main bottleneck
State preparation	control Hamiltonian	terminal fidelity	long horizons
Gate design	pulse shape or calibration knob	gate fidelity	hardware samples
Circuit design	discrete gate edits	energy or compilation cost	combinatorial search
Feedback and QEC	online correction	survival or logical success	partial observability
Metrology	adaptive measurement	information gain or error	posterior uncertainty

What is different from textbook RL?

Quantum laboratories are usually partially observed, stochastic control problems.

- ▶ The physical state may be $|\psi_t\rangle$ or ρ_t , but the agent sees O_t .
- ▶ Measurements are probabilistic and can disturb the state.
- ▶ Rewards are often terminal: fidelity, energy, lifetime, or logical success.
- ▶ The best action may depend on the entire measurement record.

$$a_t \sim \pi_\theta(\cdot \mid o_1, o_2, \dots, o_t).$$

Translation table: RL objects in quantum language

Most design mistakes start with an ambiguous definition of state and observation.

RL object	Quantum meaning	Example
State S_t	complete physical state	ρ_t in a simulator
Observation O_t	measured information	syndrome, expectation value
Action A_t	control operation	pulse, gate, measurement choice
Reward R_{t+1}	task score	fidelity, energy, survival
Episode	full protocol	one pulse sequence or QEC cycle

In hardware, formulate the problem from the data stream that is actually available.

Three environment regimes

The same policy notation hides very different experimental realities.

Regime	Data source	Practical issue
Simulated	numerical time evolution	state access may be rich, but model bias is a risk
Real hardware	the experimental device	samples are slow, noisy, and drifting
Hybrid	simulation plus device data	transfer and safe fine-tuning dominate

The central question is not only what the quantum system does, but what the policy can observe.

Markov state versus measurement record

Quantum dynamics can be Markovian in the state, while the policy input is not.

Complete simulated state

$$S_t = \rho_t, \quad \rho_{t+1} = \mathcal{T}_{a_t}(\rho_t).$$

Experimental observation

$$O_t \in \{\text{counts, records, syndromes}\}.$$

For feedback, QEC, and sensing, the policy is usually better written as

$$a_t \sim \pi_\theta(\cdot \mid h_t), \quad h_t = (o_1, a_1, \dots, o_t).$$

Model-free does not mean model-free physics

It means the update rule does not require differentiating a known transition model.

$$\rho_{t+1} = \mathcal{E}_{a_t}(\rho_t), \quad |\psi_{t+1}\rangle = U(a_t)|\psi_t\rangle.$$

- ▶ The simulator or device still obeys physics.
- ▶ The agent treats the system as an interaction source.
- ▶ Model-free training can reduce bias from an inaccurate Hamiltonian or noise model.
- ▶ The price is sample inefficiency, especially on hardware.

Algorithm choice: start from physics constraints

The algorithm is a numerical instrument; the physics defines its interface.

Constraint	Consequence
Discrete actions	Q-learning, DQN, tree search, categorical policies
Continuous pulses	policy gradients, actor-critic, evolutionary strategies
Sparse terminal reward	reward shaping, curriculum, model-based rollouts
Partial observation	belief states, recurrence, filtering, history windows
Few hardware shots	surrogates, uncertainty estimates, cautious exploration
Online feedback	low-latency policy, compact network, robust training

Directly optimize the policy that chooses experimental actions.

$$J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{T-1} \gamma^t R_{t+1} \right],$$

$$\nabla_{\theta} J(\theta) \approx \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | h_t) \hat{G}_t.$$

- ▶ Natural for continuous pulse amplitudes and stochastic policies.
- ▶ Baselines and critics reduce variance.
- ▶ PPO-like updates are common because they limit abrupt policy changes.

Value-function methods

Learn which action is promising in a given observed situation.

$$Q^{\pi}(h, a) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid H_t = h, A_t = a \right].$$

- ▶ Attractive for discrete choices: gate edits, measurement bases, corrections.
- ▶ Off-policy data reuse can matter when experiments are costly.
- ▶ Large action spaces and sparse rewards make stabilization difficult.

Good match	Less natural
QEC corrections, circuit compilation, discrete control grids	raw continuous waveform synthesis without discretization

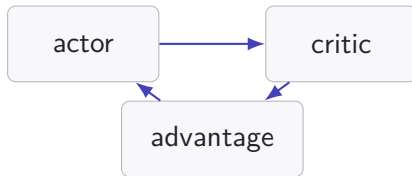
Actor-critic methods

A policy is trained with help from an estimated value function.

- ▶ The actor proposes actions.
- ▶ The critic estimates whether the outcome is better than expected.
- ▶ The advantage tells the actor which samples were useful.
- ▶ In quantum tasks the critic helps with long-horizon credit assignment.

Advantage estimate

$$A(h, a) = Q(h, a) - V(h).$$



Deep RL architectures for quantum tasks

The network type should match the structure of the data.

Input structure	Natural architecture
few scalar observables	multilayer perceptron
lattice or detector grid	convolutional network
hardware graph or Tanner graph	graph neural network
measurement history	recurrent network or transformer
candidate circuits	sequence model or tree search

The network is an inductive bias, not a neutral implementation detail.

Reward design in quantum applications

A reward is a compressed physics objective. Compression can hide failure modes.

Reward	Meaning	Hidden risk
$ \langle \psi^* \psi_T \rangle ^2$	target-state preparation	blind to path constraints
gate fidelity	operation close to target	leakage and drift may be missed
$-\langle H \rangle$	variational ground-state search	local minima or barren plateaus
survival probability	feedback stabilization	may reward trivial freezing
logical failure rate	QEC decoding	rare-event statistics are expensive
information gain	metrology	posterior depends on the model

Quantum state preparation

- ▶ Drive a quantum system from an initial state to a target state.
- ▶ The reward is often a terminal fidelity.
- ▶ The core difficulty is long-horizon credit assignment under constrained dynamics.

State preparation as finite-horizon control

A simple controlled Hamiltonian is

$$H(t) = H_0 + \sum_{k=1}^m u_k(t) H_k.$$

The objective is often

$$R_T = |\langle \psi_{\text{target}} | \psi_T \rangle|^2.$$



This is the cleanest bridge from quantum optimal control to RL.

The RL formulation of state preparation

RL object	State-preparation meaning
State or observation	density matrix, features, or measurement record
Action	next control amplitude, phase, detuning, or rotation
Reward	final fidelity, overlap, or observable proxy
Episode	one complete control protocol

$$U(a_T) \cdots U(a_2)U(a_1)|\psi_0\rangle \approx |\psi^*\rangle.$$

Few-body state preparation

Small Hilbert spaces make the physics visible but still contain the RL difficulty.

- ▶ A value-based agent can search a discrete protocol tree.
- ▶ A policy-gradient agent can tune continuous amplitudes.
- ▶ A model-based planner can exploit a trusted Hamiltonian.
- ▶ A hardware-trained policy may absorb calibration errors that the clean model misses.

Pedagogical example

A qubit or few-level oscillator where the learner chooses rotations or drive segments and receives the final target-state fidelity.

Many-body state preparation

The Hilbert space grows exponentially, so the policy must exploit structure.

- ▶ The action may be a quench, a ramp, a piecewise-constant control, or a smooth pulse family.
- ▶ Full fidelity is often unavailable; local observables or energy can be used as proxies.
- ▶ RL can find nonadiabatic shortcuts, but the result must be checked for robustness.
- ▶ Generalization across sizes requires locality, symmetry, or tensor-network structure.

Discrete versus continuous control parametrizations

The action space determines which learning algorithms are reasonable.

Protocol type	Action representation	Strength	Cost
Discrete switching	choose from a finite set	simple value learning	many steps may be needed
Piecewise constant	amplitudes per segment	close to pulse hardware	continuous optimization
Smooth pulse family	few global parameters	low dimension	less expressive
Closed-loop adaptive	condition on record history	handles noise online	latency matters

$$a_t \in \mathcal{A}_{\text{disc}} \quad \text{or} \quad a_t = (u_1(t), u_2(t), \dots, u_m(t)).$$

Observable rewards versus fidelity rewards

In many-body systems, the mathematically clean reward may be experimentally expensive.

Reward type	Advantage	Limitation
state fidelity	directly matches the target	requires strong state access or tomography
energy	often measurable locally	may not distinguish states within a phase
order parameter	experimentally accessible	can miss unwanted excitations
survival or lifetime	natural under noise	may encourage conservative controls

Always evaluate the final physical objective separately from the training reward.

A high simulated fidelity is not yet an experimental protocol.

Curriculum

Start from short horizons, small systems, or weak noise. Then increase difficulty gradually.

Robustness

Train across a distribution of Hamiltonian parameters, noise strengths, and initial conditions.

- ▶ Robust policies are usually less sharp than simulator-optimal pulses.
- ▶ Transfer tests should change both the noise and the calibration model.

Pitfalls in state-preparation RL

- ▶ Reward hacking: the agent exploits the simulator, discretization, or readout proxy.
- ▶ Lack of robustness: small calibration errors destroy a high-fidelity pulse.
- ▶ Tomography burden: estimating the reward may require many shots.
- ▶ Drift: the environment changes while the policy is being trained.
- ▶ Scaling: a policy trained on small systems may not transfer without an inductive bias.

Pulse-shape engineering for quantum gates

- ▶ The target is a unitary operation rather than a state.
- ▶ The agent searches over implementable pulses and calibration updates.
- ▶ Hardware experiments are central because real devices contain crosstalk, leakage, and drift.

The episode is a waveform; the score is how close the achieved channel is to the target gate.

$$U^* \longrightarrow U_{\theta}(T), \quad R \approx F_{\text{gate}}.$$

Object	Gate-design meaning
Observation	calibration data, tomography estimates, previous pulse segment
Action	amplitude, duration, phase, detuning, or pulse coefficient
Reward	gate fidelity, benchmarking score, leakage penalty
Episode	one full waveform implementation and measurement batch

For one qubit, the challenge is precision and robustness.

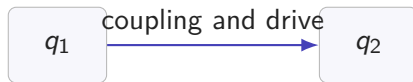
$$H(t) = \frac{1}{2} [\Delta(t)\sigma_z + \Omega_x(t)\sigma_x + \Omega_y(t)\sigma_y].$$

- ▶ Actions can be pulse samples or coefficients in a pulse basis.
- ▶ Rewards can combine gate fidelity, leakage, pulse length, and bandwidth.
- ▶ The learned pulse must respect what the waveform generator can synthesize.

Two-qubit gates

Entangling gates expose the parts of the device that are hardest to model.

- ▶ The pulse must implement the intended interaction while suppressing unwanted transitions.
- ▶ Leakage, crosstalk, and calibration drift enter directly into the reward.
- ▶ The reward is usually estimated from measurements, not from direct access to $U(T)$.



Experimental gate lesson

The review highlights superconducting-gate experiments where RL was used to optimize operational gate performance.

Reported comparison	Interpretation
learned pulse improves an operational fidelity measure	the policy adapts to device-specific imperfections
performance tested after time delay	checks sensitivity to drift
benchmarking sequences used for evaluation	the metric is tied to how the gate is used

The point is not that RL is magic search; it is that training can include the actual device.

Reward functions for gate design

A single scalar objective should reflect both the gate and the safe operating region.

Term	Role in the objective
F_{gate}	closeness to target operation
L_{leak}	penalize population outside the computational subspace
E_{pulse}	discourage heating and saturation
B_{pulse}	enforce implementable waveforms

$$R = F_{\text{gate}} - \lambda_L L_{\text{leak}} - \lambda_E E_{\text{pulse}} - \lambda_B B_{\text{pulse}}.$$

A pulse that is optimal in a simulator can fail for the same reasons RL is attractive.

Model mismatch

Hamiltonian truncations, neglected levels, calibration drift, crosstalk, and unknown transfer functions.

- ▶ Keep hard physical constraints in the action parametrization.
- ▶ Estimate reward uncertainty, not only point values.

Safer transfer

Pre-train on a family of randomized models, then fine-tune on the device with cautious exploration.

Hardware rewards are estimated, not handed to the agent exactly.

- ▶ Tomography is informative but expensive.
- ▶ Randomized benchmarking gives operational information but averages over details.
- ▶ Sparse terminal rewards force the learner to spend many shots before seeing a clear signal.
- ▶ Good baselines include calibrated optimal-control pulses and Bayesian optimization.

Quantum circuit design

- ▶ Actions are discrete symbolic edits to a circuit.
- ▶ The search space is combinatorial.
- ▶ The physics enters through gate sets, hardware connectivity, noise, and objective functions.

Circuit design as sequential program synthesis

The agent builds a quantum program one decision at a time.

$$C_0 = I, \quad C_{t+1} = G(a_t)C_t.$$

RL object	Circuit-design meaning
Observation	current circuit, cost, architecture graph, commutation features
Action	append gate, choose qubits, insert SWAP, simplify, terminate
Reward	energy, fidelity, compilation cost, depth penalty
Episode	one candidate circuit or compilation path

Action spaces for circuits

A small change in the action vocabulary changes the size and bias of the search.

Action vocabulary	What the agent learns	Trade-off
gate type only	choose from a fixed layer template	easy but restrictive
gate plus qubits	operation and placement	larger branching factor
block insertion	reusable circuit motifs	strong prior
rewrite rule	simplify or reroute a circuit	suited to compilation
terminate action	circuit length	delayed reward

$$R = R_{\text{task}} - \lambda d - \mu N_{2q}.$$

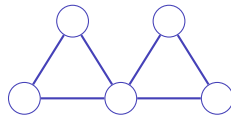
The learner searches over circuit structure before or during parameter optimization.

Step	What happens
Observe	current ansatz, energy estimate, gradients if available
Act	add an excitation, entangler, rotation layer, or symmetry-preserving block
Optimize	classical inner loop estimates the energy
Reward	energy decrease minus depth and measurement cost

$$E(\theta, C) = \langle 0 | C^\dagger(\theta) H C(\theta) | 0 \rangle.$$

The agent must obey the native gates and the processor connectivity graph.

- ▶ Equivalent circuits can have different noise costs.
- ▶ SWAP insertion makes routing a sequential problem.
- ▶ The action mask should forbid illegal gates.



A mathematically shorter circuit can be worse if it uses unreliable two-qubit gates.

RL can learn circuits that create, remove, or reroute entanglement under hardware constraints.

- ▶ State-preparation view: build a target entangled state.
- ▶ Disentangling view: map an entangled state toward a simpler product state.
- ▶ Compilation view: replace an expensive circuit by a shallower equivalent.
- ▶ Hardware view: prefer entangling operations that match the available couplings.

Why circuit RL is hard

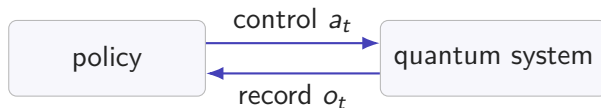
Issue	Why it matters
Huge branching factor	gate type $\times$ qubit choice $\times$ parameter choice
Delayed reward	good prefixes and bad prefixes can be hard to distinguish
Equivalence classes	many circuits implement the same unitary
Noise dependence	the best mathematical circuit may be poor on hardware
Measurement cost	estimating energy or fidelity can dominate training
Interpretability	learned circuits still need simplification and checks

Quantum feedback control

- ▶ The agent acts while the quantum system evolves.
- ▶ Measurement outcomes are part of the state of knowledge.
- ▶ Partial observability is not a nuisance here; it is the problem itself.

Feedback is a natural RL problem

A feedback controller responds to a measurement record, not to a known trajectory.



- ▶ Measurement backaction changes the hidden state.
- ▶ The controller has finite latency and bandwidth.
- ▶ The reward is usually stabilization, survival, or event suppression.

Measurement backaction changes control

Classical control intuition	Quantum feedback complication
measure the state, then act	measurement changes the state
noise is external	shot noise and backaction enter the record
state can often be estimated	coherences and leakage may remain hidden
reward can be dense	success may only be known at the end

$$h_t = (o_1, a_1, o_2, a_2, \dots, o_t).$$

Quantum trajectories and belief states

A filter converts noisy records into a classical estimate of an unobserved state.

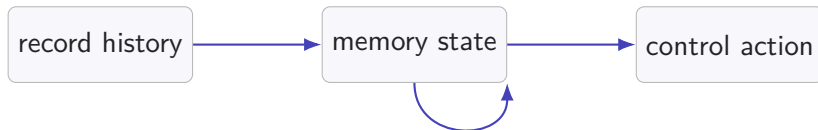
$$dy_t = \text{Tr}[(c + c^\dagger)\rho_t] dt + dW_t,$$

$$\rho_t \longrightarrow \rho_{t+dt} \quad \text{conditioned on } dy_t.$$

- ▶ The policy can act on a belief state, raw history, or learned features.
- ▶ Separating estimation and control is useful but not required.
- ▶ Real-time feedback requires inference faster than the relevant decoherence time.

Recurrent policies for feedback

A recurrent network is a compact way to carry information about the hidden quantum state.

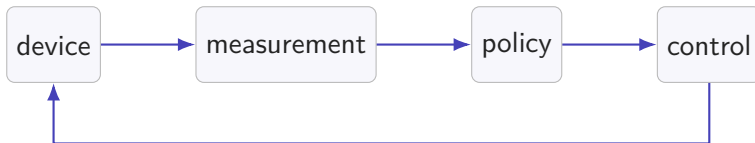


- ▶ Recurrence is not decoration; it represents accumulated information.
- ▶ Learned memory must remain stable under drift and changed noise statistics.

Experimental feedback

The review highlights real-time neural feedback controllers for quantum devices.

- ▶ The relevant architecture includes measurement electronics and control hardware.
- ▶ A simulated policy must be compiled into a low-latency controller.
- ▶ The metric should be a physical one: stabilization, reset, survival, or correction success.



Reward shaping for feedback

Dense rewards can stabilize training, but can also change the problem.

Objective	Reward example	Possible trap
keep a qubit excited	population at each step	suppresses useful measurements
stabilize an oscillator state	distance to target manifold	penalizes useful transients
protect a code space	syndrome-free intervals	misses logical errors
detect a jump quickly	early detection reward	false positives may be cheap

Evaluate the final physical metric separately from the shaped training reward.

Quantum error correction

- ▶ QEC is an online inference-and-control problem.
- ▶ The agent observes syndromes, not the error itself.
- ▶ The action must prevent logical failure before the encoded information is lost.

QEC as a partially observed decision process

The hidden state is the accumulated physical error; the observation is the syndrome.

$$E_t \text{ hidden,} \quad s_t = He_t, \quad c_t \in \mathcal{C}.$$

RL object	QEC meaning
State	true data-qubit error and measurement-error history
Observation	syndrome bits over one or more rounds
Action	Pauli correction, recovery class, or decoder decision
Reward	no logical failure, reduced residual error, longer memory time
Episode	many QEC cycles under a stochastic noise process

Decoder design choices

The policy can decode directly or learn a component inside a larger decoder.

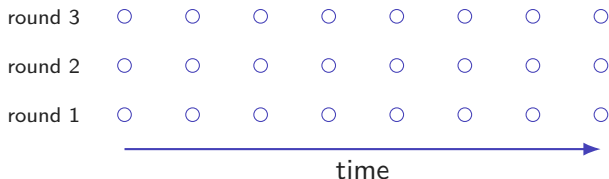
Design	Action	Strength	Risk
Direct correction	choose Pauli corrections	end-to-end learning	large action space
Syndrome clearing	local moves	local credit assignment	can create logical operators
Assist MWPM	learn weights or priors	uses decoder structure	less autonomous
Multi-agent	local agents coordinate	scales with geometry	coordination problem
GNN decoder	message passing	uses code graph	latency and generalization

A decoder is useful only if it is fast enough for the correction deadline.

- ▶ Latency: inference must finish within the QEC cycle or before feed-forward is needed.
- ▶ Throughput: the decoder must process many syndrome bits as the code grows.
- ▶ Reliability: rare logical failures are the events that matter.
- ▶ Deployment: simulated syndromes do not capture every hardware correlation.

Syndrome histories

With measurement errors, one syndrome snapshot is not enough.



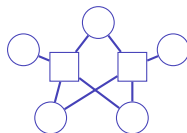
Recurrent networks, temporal windows, and space-time graph decoders all encode this history dependence.

Graph policies for QEC codes

The Tanner graph gives the policy a useful inductive bias.

$$H_X Z = s_X, \quad H_Z X = s_Z.$$

- ▶ Variable nodes are qubits.
- ▶ Check nodes are stabilizer measurements.
- ▶ Message passing can represent local syndrome correlations.
- ▶ LDPC and bivariate-bicycle codes are naturally graph-structured.



Discovery of error-correcting codes

RL can also search over the code or measurement schedule, not only the decoder.

Search object	Example action
Stabilizer generators	add, remove, or modify a check
Connectivity	place checks on a hardware graph
Gauge choices	choose measurement schedule or gauge fixing
Family parameters	balance distance, rate, check weight, and depth

A generated code still needs theoretical analysis: commutation, distance, threshold, decoding, and layout.

Evaluation in QEC is rare-event statistics

Good performance means a low logical failure rate, not a low training loss.

Metric	What it tests
Residual error weight	whether local physical errors are reduced
Syndrome clearing	whether stabilizer violations disappear
Logical failure rate	whether encoded information survives
Threshold or pseudo-threshold	scaling with code distance and physical error rate
Latency	whether decoding can run in real time

$$p_{\text{succ}} = \Pr(\text{no logical error after } T \text{ rounds}).$$

When the agent itself uses quantum resources

- ▶ Do not confuse RL controlling a quantum system with quantum resources inside the learner.
- ▶ The review treats quantum RL separately from the main control applications.
- ▶ The clean example is amplitude amplification over actions.

Classical RL on quantum systems versus QRL

Let C mean classical and Q mean quantum.

Agent	Environment	Interpretation
C	C	ordinary reinforcement learning
C	Q	classical RL applied to a quantum task
Q	C	quantum-enhanced reinforcement learning
Q	Q	fully quantum reinforcement learning

Most quantum-technology applications in this lecture are in the (C, Q) row.

Quantum RL can encode actions and rewards in separate registers.

$$\mathcal{H} = \mathcal{H}_A \otimes \mathcal{H}_R, \quad U_E |a\rangle_A |0\rangle_R = |a\rangle_A |r(a)\rangle_R.$$

- ▶ A classical episode samples one action and measures one reward.
- ▶ A coherent episode can query actions in superposition.
- ▶ The reward qubit can phase-mark winning actions.

Prepare the action register in a two-action superposition,

$$|\psi\rangle_A = \cos \xi |\ell\rangle_A + \sin \xi |w\rangle_A.$$

Prepare the reward register as $|-\rangle_R = (|0\rangle_R - |1\rangle_R)/\sqrt{2}$. Then

$$U_E |\psi\rangle_A |-\rangle_R = (\cos \xi |\ell\rangle_A - \sin \xi |w\rangle_A) |-\rangle_R.$$

The environment has acted as a phase oracle for rewarding actions.

Amplitude amplification in the RL setting

A reflection about the initial action state increases the weight of rewarded actions.

$$U_{\text{ref}} = 2|\psi\rangle\langle\psi| - I.$$

- ▶ This is Grover-like amplification over actions.
- ▶ It can help early exploration in search-like tasks.
- ▶ It does not solve partial observability, noisy rewards, or credit assignment by itself.

The review uses photon exchange as a concrete example of quantum-enhanced learning.

- ▶ Classical episodes prepare one action and receive a binary reward.
- ▶ Quantum episodes use coherent superposition and phase marking.
- ▶ The quantum strategy can increase average reward quickly at early times.
- ▶ A stable strategy still needs classical reward measurement and updating.

Why combine quantum and classical strategies?

Amplitude amplification can overshoot if applied too long.

Early stage

Quantum episodes amplify rewarding actions and improve exploration.

Later stage

Classical episodes provide measured rewards for stable policy updates.

The useful lesson is hybrid: quantum resources can change sampling, while classical learning supplies stable adaptation.

Quantum policies and value circuits

Another route is to represent a policy or value function with a variational quantum circuit.

$$\pi_{\theta}(a \mid o) \longleftrightarrow \text{measure } U_{\theta}(o)|0 \cdots 0\rangle.$$

Ingredient	Role
data encoding	map observations to a quantum state or circuit parameters
variational circuit	trainable policy or value approximator
measurement	sample an action or estimate a value
classical optimizer	update parameters from rewards

Claims of advantage require careful resource accounting.

- ▶ A coherent quantum query is not the same resource as one classical laboratory episode.
- ▶ Amplitude amplification helps search-like subproblems, not every RL problem.
- ▶ Variational quantum policies can be compact, but compactness is not an advantage proof.
- ▶ Comparisons need strong classical baselines and the same sample budget.

Quantum metrology and adaptive sensing

- ▶ The task is to estimate an unknown parameter.
- ▶ Actions choose probes, controls, or measurement bases.
- ▶ Rewards can be information gain, posterior-variance reduction, or final estimation accuracy.

Metrology as adaptive experiment design

An RL policy can choose the next measurement based on what it already knows.

RL object	Metrology meaning
Hidden state	unknown parameter ϕ or field profile
Observation	measurement outcome conditioned on the current probe
Action	probe time, control phase, measurement basis, feedback operation
Reward	information gain, reduced posterior variance, low final error

$$a_t \sim \pi_\theta(\cdot \mid p_t(\phi)),$$

where $p_t(\phi)$ denotes the current posterior or its sufficient statistics.

Reward choices for sensing

Reward	Policy tendency	Comment
negative posterior variance	reduce uncertainty	natural Bayesian objective
Fisher information	choose locally informative settings	may be locally biased
final squared error	optimize end-to-end estimation	terminal and sparse
classification success	distinguish hypotheses	useful for decision tasks
cost-sensitive reward	balance precision against time or energy	important under decoherence

Changing the prior distribution changes the RL problem.

A method that ignores laboratory bottlenecks is not a quantum-technology method.

Reality	Consequence for RL
shots are expensive	reuse data and quantify uncertainty
noise drifts	monitor policy degradation and train robustly
measurement is invasive	observation design is part of the physics
latency is finite	feedback policies must be compact and compiled
safety constraints exist	constrain actions before reward evaluation
simulators are approximate	evaluate transfer and fine-tune carefully

Design checklist for a quantum-RL project

Question	Answer before training
What is actually observed?	not the wavefunction unless this is simulator-only
What actions are physically allowed?	amplitude, bandwidth, latency, gate set, connectivity
How is reward estimated?	shots, tomography, benchmarking, logical trials, posterior updates
What is the baseline?	optimal control, decoder, transpiler, Bayesian estimator, threshold policy
How will transfer be tested?	noise randomization, drift tests, hardware fine-tuning
What failure mode matters?	leakage, logical operator, instability, unsafe pulse, wrong prior

- ▶ RL is a language for sequential interaction with quantum systems.
- ▶ The most mature applications are classical agents controlling quantum devices.
- ▶ Feedback and QEC are naturally partially observed decision problems.
- ▶ QRL adds quantum resources inside the agent and should be kept conceptually separate.

Primary reading

Bukov and Marquardt, *Reinforcement Learning for Quantum Technology*, arXiv:2601.18953.

Background

Sutton and Barto for RL;
quantum-control and open-system
notes for Hamiltonian control,
measurement, and noise.